



HCM-UTE

TRƯỜNG ĐẠI HỌC

CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

HCMC University of Technology and Engineering

[SOEN330679] CÔNG NGHỆ PHẦN MỀM

CHƯƠNG 3: THIẾT KẾ DỮ LIỆU (01)

ThS. Võ Quang Phúc

MỤC TIÊU

1. **MT1.** Giải thích được vai trò và vị trí của thiết kế dữ liệu trong quy trình phát triển phần mềm hiện đại.
2. **MT2.** Phân biệt được các mô hình lưu trữ dữ liệu phổ biến (Relational, Document, Key-Value, Graph, File, Cloud Storage) và biết khi nào nên dùng mô hình nào.
3. **MT3.** Vẽ được Sơ đồ lớp (Class Diagram) theo chuẩn UML 2.x cho một bài toán nghiệp vụ vừa và nhỏ.
4. **MT4.** Vẽ được sơ đồ quan hệ thực thể (ERD) ở mức conceptual và logical cho domain của bài tập lớn.
5. **MT5.** So sánh được Class Diagram và ERD; nhận biết khi nào sử dụng công cụ nào trong dự án thực tế.

NỘI DUNG

1. Tổng quan về thiết kế dữ liệu trong CNPM
2. Các mô hình lưu trữ dữ liệu hiện đại
3. Sơ đồ lớp tổng quát (UML Class Diagram)
4. Mô hình quan hệ thực thể (ERD)
5. Mối liên hệ giữa Class Diagram và ERD
6. Ví dụ minh họa (Quản lý quán cafe)
7. Thực hành

1. Tổng quan về thiết kế dữ liệu trong CNPM

1.1. Định nghĩa:

- ✓ Thiết kế dữ liệu (Data Design) là hoạt động chuyển đổi mô hình thông tin được tạo ra trong giai đoạn phân tích yêu cầu thành các cấu trúc dữ liệu sẽ được sử dụng để hiện thực hóa phần mềm.
- ✓ Mục tiêu là đảm bảo dữ liệu được tổ chức một cách có hệ thống, dễ truy xuất, dễ bảo trì và đáp ứng được các yêu cầu phi chức năng như hiệu năng, bảo mật, khả năng mở rộng.

1. Tổng quan về thiết kế dữ liệu trong CNPM

1.1. Định nghĩa:

Trong các phương pháp luận hiện đại (Domain-Driven Design – DDD, Clean Architecture), thiết kế dữ liệu không chỉ là vẽ ERD hay tạo bảng cơ sở dữ liệu mà còn bao gồm việc:

- ✓ Xác định các thực thể nghiệp vụ (Domain Entities) và các giá trị (Value Objects).
- ✓ Thiết kế ranh giới giữa các phần của hệ thống (Bounded Contexts).
- ✓ Lựa chọn mô hình lưu trữ phù hợp với từng phần (polyglot persistence).
- ✓ Định nghĩa các quy tắc nghiệp vụ (Business Rules) gắn liền với dữ liệu.

1. Tổng quan về thiết kế dữ liệu trong CNPM

1.2. Vị trí trong quy trình phát triển phần mềm

Thiết kế dữ liệu là một trong ba thành phần của thiết kế phần mềm, bên cạnh thiết kế xử lý (chức năng) và thiết kế giao diện. Trong các mô hình phát triển khác nhau, vị trí của hoạt động này có sự khác biệt.

Mô hình phát triển	Vị trí của thiết kế dữ liệu
Waterfall	Một giai đoạn riêng biệt, làm sau khi hoàn thành phân tích yêu cầu
Iterative / Incremental	Lặp lại nhiều lần, mỗi vòng lặp tinh chỉnh thiết kế dữ liệu
Agile / Scrum	Diễn ra liên tục trong từng sprint, có thể thay đổi theo phản hồi
DevOps / CI-CD	Đi kèm với migration scripts được version control

1. Tổng quan về thiết kế dữ liệu trong CNPM

1.3. Ba mức thiết kế dữ liệu

Thiết kế dữ liệu được phân thành ba mức trừu tượng:

- (1) Mức quan niệm (**Conceptual Level**): Mô tả dữ liệu ở góc độ nghiệp vụ, không phụ thuộc vào công nghệ. Công cụ phổ biến: ERD mức conceptual, Class Diagram domain model. Đối tượng đọc: khách hàng, chuyên viên phân tích nghiệp vụ.
- (2) Mức logic (**Logical Level**): Mô tả cấu trúc dữ liệu chi tiết hơn, đã chọn loại mô hình (quan hệ, hướng đối tượng, document...) nhưng chưa gắn với hệ quản trị cụ thể. Công cụ: ERD mức logical, schema design.
- (3) Mức vật lý (**Physical Level**): Mô tả cách dữ liệu được lưu trữ thật sự trên hệ quản trị cụ thể, bao gồm kiểu dữ liệu, ràng buộc, chỉ mục, phân vùng

1. Tổng quan về thiết kế dữ liệu trong CNPM

1.4. Tại sao thiết kế dữ liệu lại quan trọng?

Trong thực tế, chi phí sửa lỗi thiết kế dữ liệu sau khi hệ thống đã đi vào vận hành có thể cao gấp 10–100 lần so với việc thiết kế đúng từ đầu. Lý do:

- ✓ Dữ liệu là tài sản tồn tại lâu dài hơn cả mã nguồn. Một schema đặt sai có thể kéo theo việc viết lại nhiều module.
- ✓ Migration (chuyển đổi schema) trên hệ thống đang chạy production luôn tiềm ẩn rủi ro mất dữ liệu.
- ✓ Thiết kế dữ liệu kém dẫn đến truy vấn chậm, ảnh hưởng trực tiếp đến trải nghiệm người dùng.
- ✓ Trong kỷ nguyên dữ liệu lớn, các quyết định ở giai đoạn này ảnh hưởng đến khả năng phân tích, báo cáo, AI/ML sau này.

2. Các mô hình lưu trữ dữ liệu hiện đại

- ❑ Trong các năm 1980–2000, gần như mọi ứng dụng đều sử dụng cơ sở dữ liệu quan hệ (RDBMS) làm lựa chọn mặc định.
- ❑ Từ năm 2009 trở đi, sự bùng nổ của các ứng dụng web quy mô lớn (Facebook, Twitter, Amazon...) và dữ liệu phi cấu trúc đã thúc đẩy sự ra đời của nhiều mô hình lưu trữ khác.
- ❑ Hiện nay, một hệ thống phần mềm hiện đại thường sử dụng nhiều mô hình lưu trữ kết hợp – gọi là polyglot persistence (đa hình lưu trữ).

(1) Cơ sở dữ liệu quan hệ (Relational Database – RDBMS)

(2) Cơ sở dữ liệu Document (Document Database)

(3) Cơ sở dữ liệu Key-Value

(4) Cơ sở dữ liệu đồ thị (Graph Database)

(5) Lưu trữ file và Cloud Storage

2. Các mô hình lưu trữ dữ liệu hiện đại

2.1. Cơ sở dữ liệu quan hệ (Relational Database – RDBMS)

❑ Đặc điểm:

- ✓ Dữ liệu được tổ chức thành các bảng (table) gồm các hàng (row) và cột (column).
- ✓ Quan hệ giữa các bảng được biểu diễn thông qua khóa chính (primary key) và khóa ngoại (foreign key).
- ✓ Truy vấn bằng ngôn ngữ SQL chuẩn hóa.
- ✓ Đặc tính nổi bật: tuân thủ tính chất ACID (Atomicity – Consistency – Isolation – Durability), đảm bảo tính nhất quán và toàn vẹn dữ liệu

2. Các mô hình lưu trữ dữ liệu hiện đại

2.1. Cơ sở dữ liệu quan hệ (Relational Database (RDBMS))

❑ Các hệ quản trị phổ biến:

- ✓ PostgreSQL: mã nguồn mở, mạnh mẽ, hỗ trợ tốt JSON và các tính năng nâng cao. Là một trong những lựa chọn phổ biến nhất.
- ✓ MySQL / MariaDB: phổ biến trong các ứng dụng web truyền thống, hệ sinh thái LAMP/LEMP.
- ✓ Microsoft SQL Server: sử dụng nhiều trong môi trường doanh nghiệp Microsoft.
- ✓ Oracle Database: các hệ thống lớn, ngân hàng, viễn thông.
- ✓ SQLite: cơ sở dữ liệu nhúng, dùng cho ứng dụng di động, desktop, prototype.

2. Các mô hình lưu trữ dữ liệu hiện đại

2.1. Cơ sở dữ liệu quan hệ (Relational Database (RDBMS))

❑ Khi nào nên dùng RDBMS?

- ✓ Dữ liệu có cấu trúc rõ ràng, quan hệ giữa các thực thể chặt chẽ.
- ✓ Yêu cầu cao về tính nhất quán (giao dịch tài chính, đặt vé, đặt hàng).
- ✓ Cần truy vấn phức tạp với nhiều bảng liên kết, tổng hợp, báo cáo.
- ✓ Đội phát triển có kinh nghiệm với SQL.

2. Các mô hình lưu trữ dữ liệu hiện đại

2.2. Cơ sở dữ liệu Document (Document Database)

□ Đặc điểm

- ✓ Dữ liệu được lưu dưới dạng các tài liệu (document), thường là JSON hoặc BSON.
- ✓ Mỗi tài liệu là một bản ghi tự chứa, không cần tuân theo schema cứng.
- ✓ Hai tài liệu trong cùng một collection có thể có cấu trúc khác nhau.

2. Các mô hình lưu trữ dữ liệu hiện đại

2.2. Cơ sở dữ liệu Document (Document Database)

□ Các hệ quản trị phổ biến:

- ✓ MongoDB: phổ biến nhất, mã nguồn mở, hỗ trợ tốt cho ứng dụng Node.js, Python.
- ✓ Couchbase, Amazon DocumentDB: các lựa chọn doanh nghiệp.
- ✓ Firebase Firestore: cloud-native, phổ biến trong ứng dụng di động

2. Các mô hình lưu trữ dữ liệu hiện đại

2.2. Cơ sở dữ liệu Document (Document Database)

□ Khi nào dùng Document Database?

- ✓ Dữ liệu có cấu trúc linh hoạt, thay đổi nhanh trong giai đoạn phát triển.
- ✓ Cần lưu các đối tượng phức tạp, lồng nhau (ví dụ: bài viết blog có nhiều bình luận, sản phẩm có nhiều thuộc tính tùy biến).
- ✓ Ưu tiên tốc độ phát triển hơn tính nhất quán tuyệt đối.
- ✓ Hệ thống cần mở rộng theo chiều ngang (horizontal scaling) dễ dàng

2. Các mô hình lưu trữ dữ liệu hiện đại

2.2. Cơ sở dữ liệu Document

□ Ví dụ

```
1 {  
2   "_id": "ORD-20251110-001",  
3   "ngayTao": "2025-11-10T14:30:00Z",  
4   "thuNgan": "chi_hang",  
5   "khachHang": {  
6     "type": "VIP",  
7     "maThe": "VIP-0042",  
8     "ten": "Anh Tuan"  
9   },  
10  "danhSachMon": [  
11    {  
12      "tenMon": "Cafe sua da",  
13      "soLuong": 2,  
14      "donGia": 30000  
15    },  
16    {  
17      "tenMon": "Banh croissant",  
18      "soLuong": 1,  
19      "donGia": 45000  
20    }  
21  ],  
22  "giamGia": {  
23    "loai": "VIP",  
24    "phanTram": 10  
25  },  
26  "tongTien": 94500,  
27  "thanhToan": {  
28    "phuongThuc": "QR-VNPay",  
29    "trangThai": "SUCCESS"  
30  }  
31 }
```

2. Các mô hình lưu trữ dữ liệu hiện đại

2.3. Cơ sở dữ liệu Key-Value

□ Đặc điểm:

- ✓ Đơn giản nhất trong các loại NoSQL: dữ liệu chỉ là cặp khóa – giá trị.
- ✓ Tốc độ truy xuất cực nhanh do cấu trúc đơn giản, thường được lưu trên RAM

2. Các mô hình lưu trữ dữ liệu hiện đại

2.3. Cơ sở dữ liệu Key-Value

□ Các hệ quản trị phổ biến:

- ✓ Redis: phổ biến nhất, ngoài key-value còn hỗ trợ nhiều cấu trúc dữ liệu (list, set, sorted set, hash).
- ✓ Memcached: chuyên dùng làm cache.
- ✓ Amazon DynamoDB: cloud, scale tốt.

2. Các mô hình lưu trữ dữ liệu hiện đại

2.3. Cơ sở dữ liệu Key-Value

□ Khi nào dùng key-Value?

- ✓ Cache (đệm): lưu kết quả truy vấn để giảm tải cho RDBMS.
- ✓ Quản lý session người dùng, token đăng nhập.
- ✓ Hệ thống xếp hàng (queue) đơn giản, đếm số, leaderboard.
- ✓ Real-time data: thông báo, chat, theo dõi trạng thái online.

Trong ví dụ Quán Café:

Có thể dùng Redis để cache danh sách menu món (ít thay đổi, được truy vấn liên tục) và theo dõi trạng thái đơn hàng đang chờ pha chế của barista. Đây là ứng dụng thực tế của polyglot persistence: dữ liệu chính ở PostgreSQL, cache ở Redis.

2. Các mô hình lưu trữ dữ liệu hiện đại

2.4. Cơ sở dữ liệu đồ thị (Graph Database)

□ Đặc điểm:

- ✓ Dữ liệu được lưu dưới dạng các nút (node) và cạnh (edge) thể hiện quan hệ giữa các nút.
- ✓ Tối ưu cho việc truy vấn các quan hệ phức tạp, nhiều cấp.

2. Các mô hình lưu trữ dữ liệu hiện đại

2.4. Cơ sở dữ liệu đồ thị (Graph Database)

□ Các hệ quản trị phổ biến:

- ✓ Neo4j – phổ biến nhất, có ngôn ngữ truy vấn Cypher.
- ✓ Amazon Neptune, ArangoDB.

2. Các mô hình lưu trữ dữ liệu hiện đại

2.4. Cơ sở dữ liệu đồ thị (Graph Database)

□ Khi nào nên dùng Graph DB?

- ✓ Mạng xã hội: bạn của bạn, gợi ý kết nối.
- ✓ Hệ thống gợi ý sản phẩm, bài viết.
- ✓ Phát hiện gian lận trong giao dịch tài chính.
- ✓ Quản lý tri thức (knowledge graph), bản đồ tri thức

2. Các mô hình lưu trữ dữ liệu hiện đại

2.5. Lưu trữ file và Cloud Storage

□ Đặc điểm:

- ✓ Khác với các loại trên (chủ yếu lưu dữ liệu có cấu trúc), file storage và cloud storage chuyên lưu các tệp lớn: hình ảnh, video, tài liệu PDF, file Excel...

2. Các mô hình lưu trữ dữ liệu hiện đại

2.5. Lưu trữ file và Cloud Storage

□ Các dịch vụ phổ biến

- ✓ **Amazon S3**: tiêu chuẩn de facto của ngành công nghiệp.
- ✓ **Google Cloud Storage, Azure Blob Storage**: tương tự Amazon S3, từ Google và Microsoft.
- ✓ **Cloudinary, Imgix**: chuyên cho hình ảnh, có CDN và xử lý hình tự động.
- ✓ **Firebase Storage**: dễ tích hợp với ứng dụng di động.

2. Các mô hình lưu trữ dữ liệu hiện đại

2.5. Lưu trữ file và Cloud Storage

❑ **Mô hình kết hợp:** Trong các ứng dụng thực tế hiện đại, mô hình thường gặp là:

- ✓ Metadata (thông tin mô tả) của file được lưu trong RDBMS (tên file, người tải lên, kích thước, đường dẫn).
- ✓ File thực tế được lưu trên S3 hoặc dịch vụ tương tự.
- ✓ Ứng dụng nhận URL từ DB, sau đó tải file từ cloud storage.

Trong ví dụ Quán Café:

Hình ảnh các món trong menu nên được lưu trên Clouinary hoặc Firebase Storage, còn thông tin món (tên, giá, mô tả) lưu trong RDBMS với một cột url_anh trỏ tới file trên cloud.

2. Các mô hình lưu trữ dữ liệu hiện đại

2.6. Tóm tắt

Mô hình	Hệ quản trị tiêu biểu	Ưu điểm chính	Trường hợp sử dụng
Relational (SQL)	PostgreSQL, MySQL, SQL Server	ACID, truy vấn phức tạp, chuẩn hóa	Đa số ứng dụng nghiệp vụ
Document	MongoDB, Firestore	Linh hoạt schema, luồng dữ liệu	CMS, e-commerce, IoT
Key-Value	Redis, Memcached	Cực nhanh, đơn giản	Cache, session, real-time
Graph	Neo4j	Truy vấn quan hệ nhiều cấp	Mạng xã hội, gợi ý
Cloud Storage	Amazon S3, Cloudinary	Lưu file lớn, scale vô hạn	Ảnh, video, tài liệu

2. Các mô hình lưu trữ dữ liệu hiện đại

2.7. Xu hướng

- ✓ **Polyglot Persistence:** Dùng nhiều loại DB trong cùng một hệ thống, mỗi loại cho mục đích phù hợp nhất.
- ✓ **Managed Databases (DBaaS):** Các dịch vụ quản lý DB trên cloud như AWS RDS, Google Cloud SQL, MongoDB Atlas, Supabase, PlanetScale. Đội phát triển không cần lo về backup, scaling, bảo mật vật lý.
- ✓ **Backend-as-a-Service (BaaS):** Firebase, Supabase, Appwrite cung cấp cả DB, authentication, file storage, real-time sync trong một gói. Rất phù hợp cho dự án nhỏ và MVP.

2. Các mô hình lưu trữ dữ liệu hiện đại

2.7. Xu hướng

- ✓ **ORM/ODM hiện đại:** Prisma (TypeScript/Node.js), TypeORM, Drizzle, SQLAlchemy (Python), Hibernate (Java), Entity Framework (.NET) giúp lập trình viên thao tác với DB qua đối tượng thay vì SQL thuần.
- ✓ **Database Migration tools:** Flyway, Liquibase, Prisma Migrate, Alembic giúp version control schema theo thời gian, đồng bộ với mã nguồn.
- ✓ **Quy định bảo vệ dữ liệu:** GDPR (Châu Âu), Luật An ninh mạng và Nghị định 13/2023/NĐ-CP (Việt Nam) về bảo vệ dữ liệu cá nhân yêu cầu các hệ thống phải mã hóa dữ liệu nhạy cảm, cho phép xóa dữ liệu cá nhân theo yêu cầu, ghi log truy cập

3. Sơ đồ lớp tổng quát (UML Class Diagram)

3.1. Giới thiệu

- ❑ **Sơ đồ lớp (Class Diagram)** là một trong những sơ đồ cấu trúc quan trọng nhất của UML (Unified Modeling Language) – ngôn ngữ mô hình hóa thống nhất được chuẩn hóa bởi tổ chức OMG (Object Management Group).
- ❑ Phiên bản hiện hành là UML 2.5.1.
- ❑ **Class Diagram** mô tả cấu trúc tĩnh của hệ thống thông qua các lớp (class), thuộc tính (attribute), phương thức (method) và quan hệ giữa các lớp.
- ❑ Đây là cầu nối giữa phân tích nghiệp vụ và lập trình hướng đối tượng.

3. Sơ đồ lớp tổng quát (UML Class Diagram)

3.2. Vai trò

- ✓ Là bản thiết kế trực quan cho lập trình viên khi viết code hướng đối tượng.
- ✓ Là tài liệu giao tiếp giữa các thành viên trong nhóm phát triển.
- ✓ Là cơ sở để sinh ra schema cơ sở dữ liệu thông qua ORM.
- ✓ Là nền tảng cho việc tài liệu hóa kiến trúc phần mềm.

3. Sơ đồ lớp tổng quát (UML Class Diagram)

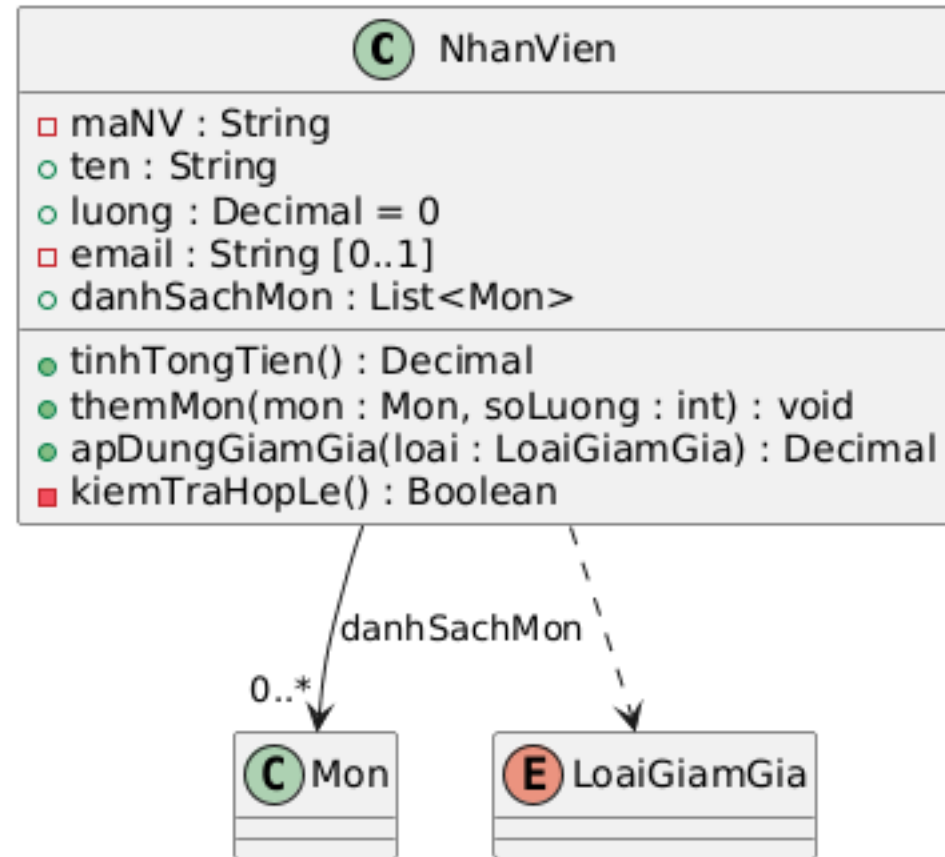
3.3. Các thành phần cơ bản

- ❑ Một lớp trong UML được biểu diễn bằng một hình chữ nhật chia làm ba phần:
 - ✓ Phần đầu: Tên lớp (ClassName), viết hoa chữ cái đầu của mỗi từ (PascalCase).
 - ✓ Phần giữa: Danh sách các thuộc tính (attributes).
 - ✓ Phần dưới: Danh sách các phương thức (methods/operations).

3. Sơ đồ lớp tổng quát (UML Class Diagram)

3.3. Các thành phần cơ bản

❑ Ví dụ



3. Sơ đồ lớp tổng quát (UML Class Diagram)

3.3. Các thành phần cơ bản

❑ Cú pháp thuộc tính

```
visibility name : type [multiplicity] = defaultValue {property}
```

Ví dụ:

```
- maNV : String
```

```
+ ten : String
```

```
+ lương : Decimal = 0
```

```
- email : String [0..1]
```

```
+ danhSachMon : List<Mon>
```

3. Sơ đồ lớp tổng quát (UML Class Diagram)

3.3. Các thành phần cơ bản

❑ Cú pháp phương thức

```
visibility name (parameter list) : returnType {property}
```

Ví dụ:

```
+ tinhTongTien() : Decimal  
+ themMon(mon: Mon, soLuong: int) : void  
+ apDungGiamGia(loai: LoaiGiamGia) : Decimal  
- kiemTraHopLe() : Boolean
```

3. Sơ đồ lớp tổng quát (UML Class Diagram)

3.3. Các thành phần cơ bản

❑ Ký hiệu phạm vi truy cập

Ký hiệu	Tên gọi	Ý nghĩa
+	public	Truy cập được từ mọi lớp
-	private	Chỉ truy cập trong nội bộ lớp
#	protected	Truy cập trong lớp và các lớp con
~	package	Truy cập trong cùng gói (package)

3. Sơ đồ lớp tổng quát (UML Class Diagram)

3.4. Các loại quan hệ giữa các lớp

Quan hệ giữa các lớp là yếu tố quan trọng nhất của Class Diagram, thể hiện cách các lớp tương tác với nhau.

- (1) Association (Liên kết)
- (2) Multiplicity (Bội số)
- (3) Aggregation (Tập hợp)
- (4) Composition (Hợp thành)
- (5) Generalization / Inheritance (Tổng quát hóa / Kế thừa)
- (6) Realization (Hiện thực hóa)
- (7) Dependency (Phụ thuộc)

3. Sơ đồ lớp tổng quát (UML Class Diagram)

3.4. Các loại quan hệ giữa các lớp

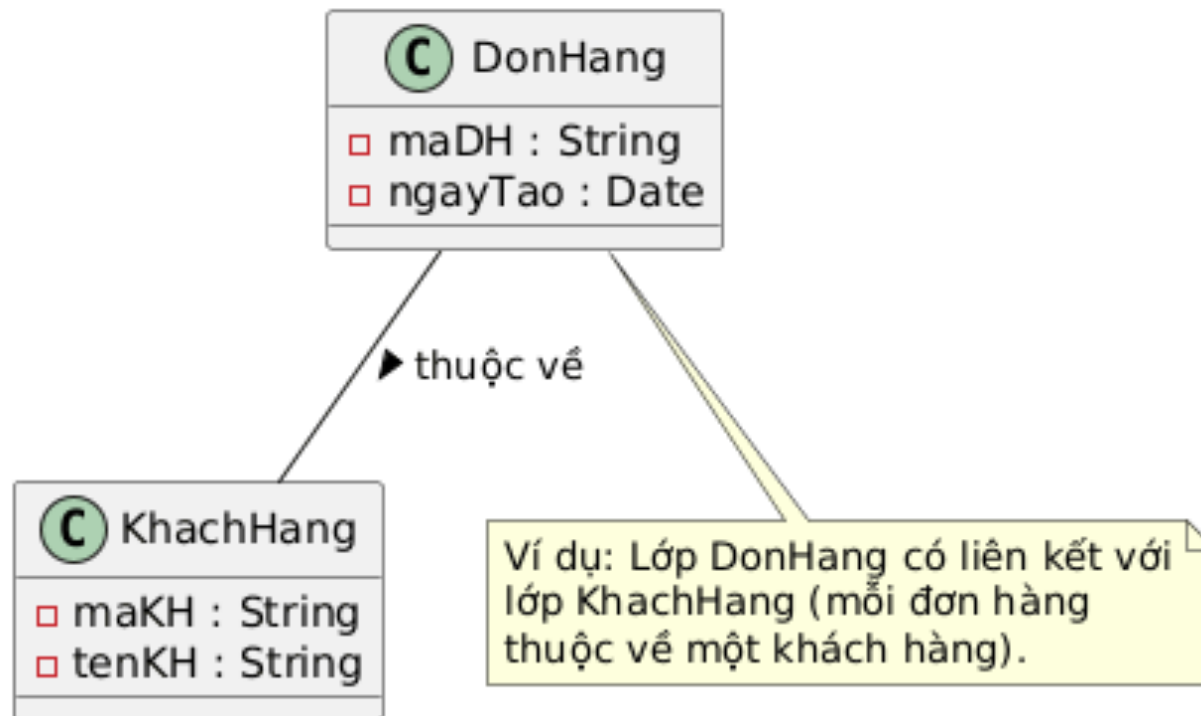
(1) Association (Liên kết):

- ✓ Quan hệ cơ bản nhất, biểu diễn rằng một lớp có liên hệ với lớp khác.
- ✓ **Ký hiệu:** đường thẳng nối giữa hai lớp.
- ✓ **Ví dụ:** Lớp DonHang có liên kết với lớp KhachHang (mỗi đơn hàng thuộc về một khách hàng).

3. Sơ đồ lớp tổng quát (UML Class Diagram)

3.4. Các loại quan hệ giữa các lớp

(1) Association (Liên kết)



3. Sơ đồ lớp tổng quát (UML Class Diagram)

3.4. Các loại quan hệ giữa các lớp

(2) Multiplicity (Bội số):

Đi kèm với association, mô tả số lượng đối tượng tham gia vào quan hệ.

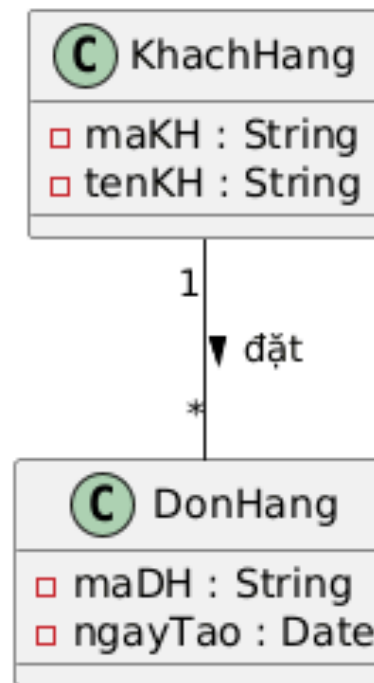
Ký hiệu	Ý nghĩa
1	Chính xác một
0..1	Không có hoặc một (tùy chọn)
*	Không giới hạn (0 hoặc nhiều)
1..*	Một hoặc nhiều
n..m	Từ n đến m

3. Sơ đồ lớp tổng quát (UML Class Diagram)

3.4. Các loại quan hệ giữa các lớp

Ví dụ: Một KháchHang (1) đặt nhiều DonHang (*), tức quan hệ 1-N (một-nhiều).

(2) Multiplicity (Bội số)



Ví dụ: Một KháchHang (1) đặt nhiều DonHang (*), tức quan hệ 1-N (một - nhiều).

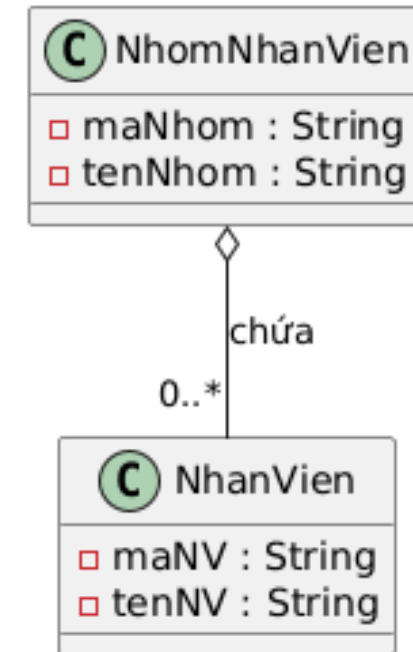
3. Sơ đồ lớp tổng quát (UML Class Diagram)

3.4. Các loại quan hệ giữa các lớp

(3) Aggregation (Tập hợp):

- ✓ Quan hệ “có” yếu, đối tượng thành phần có thể tồn tại độc lập với đối tượng chứa nó.
- ✓ **Ký hiệu:** hình thoi rỗng (◇) ở đầu lớp chứa.
- ✓ **Ví dụ:** Lớp `NhomNhanVien` chứa nhiều `NhanVien`, nhưng khi xóa nhóm thì các nhân viên vẫn tồn tại.

(3) Aggregation (Tập hợp)



Ví dụ: Lớp `NhomNhanVien` chứa nhiều `NhanVien`, nhưng khi xóa nhóm thì các nhân viên vẫn tồn tại.
(Hình thoi rỗng ở đầu lớp chứa)

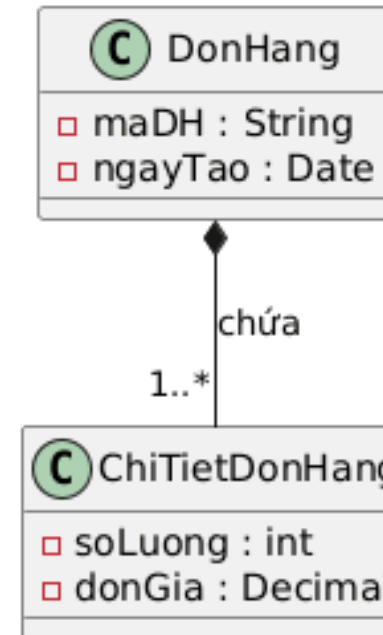
3. Sơ đồ lớp tổng quát (UML Class Diagram)

3.4. Các loại quan hệ giữa các lớp

(4) Composition (Hợp thành):

- ✓ Quan hệ “có” mạnh, đối tượng thành phần không thể tồn tại nếu không có đối tượng chứa nó.
- ✓ **Ký hiệu:** hình thoi đặc (◆) ở đầu lớp chứa.
- ✓ **Ví dụ:** Lớp DonHang chứa nhiều ChiTietDonHang. Khi xóa đơn hàng, các chi tiết đi kèm cũng bị xóa theo.

(4) Composition (Hợp thành)



Ví dụ: Lớp DonHang chứa nhiều ChiTietDonHang. Khi xóa đơn hàng, các chi tiết đi kèm cũng bị xóa theo. (Hình thoi đặc ở đầu lớp chứa)

3. Sơ đồ lớp tổng quát (UML Class Diagram)

3.4. Các loại quan hệ giữa các lớp

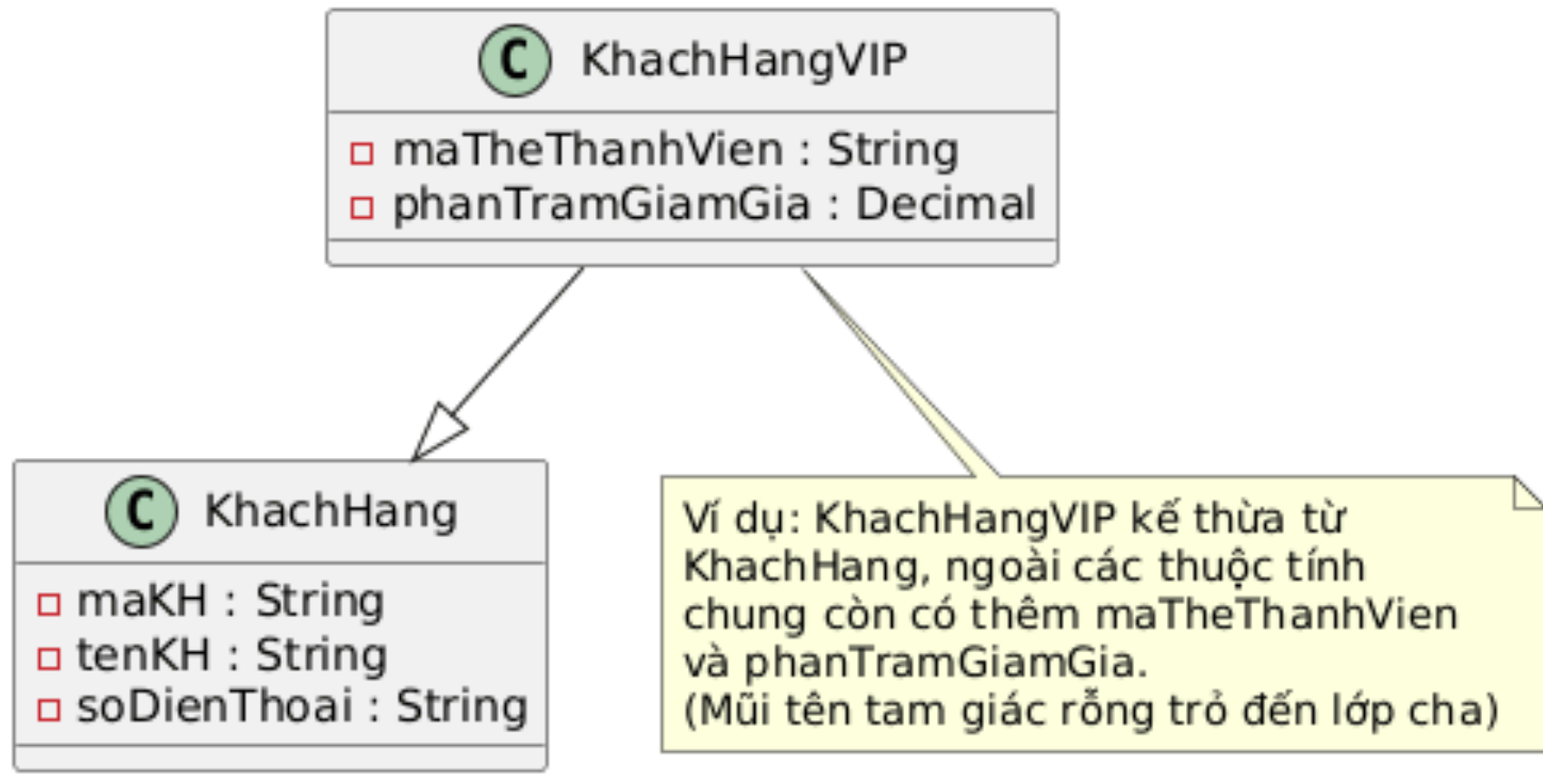
(5) Generalization / Inheritance (Tổng quát hóa / Kế thừa):

- ✓ Quan hệ cha-con: lớp con kế thừa thuộc tính và phương thức từ lớp cha.
- ✓ **Ký hiệu:** mũi tên tam giác rỗng (Δ) trỏ từ lớp con đến lớp cha.
- ✓ **Ví dụ:** `KhachHangVIP` kế thừa từ `KhachHang`, ngoài các thuộc tính chung còn có thêm thuộc tính *maTheThanhVien* và *phanTramGiamGia*.

3. Sơ đồ lớp tổng quát (UML Class Diagram)

3.4. Các loại quan hệ giữa các lớp

(5) Generalization / Inheritance (Tổng quát hóa / Kế thừa)



3. Sơ đồ lớp tổng quát (UML Class Diagram)

3.4. Các loại quan hệ giữa các lớp

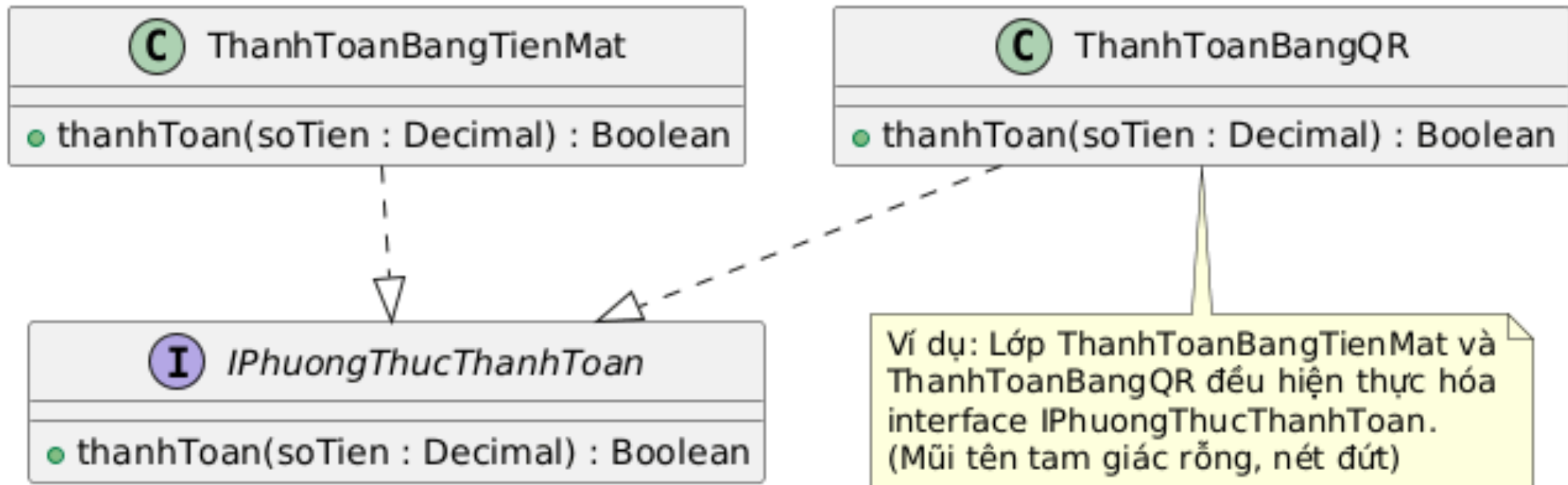
(6) Realization (Hiện thực hóa):

- ✓ Quan hệ giữa interface và lớp triển khai interface đó.
- ✓ **Ký hiệu:** mũi tên tam giác rỗng nét đứt.
- ✓ **Ví dụ:** Lớp ThanhToanBangTienMat và ThanhToanBangQR đều hiện thực hóa interface IPhuongThucThanhToan

3. Sơ đồ lớp tổng quát (UML Class Diagram)

3.4. Các loại quan hệ giữa các lớp

(6) Realization (Hiện thực hóa)

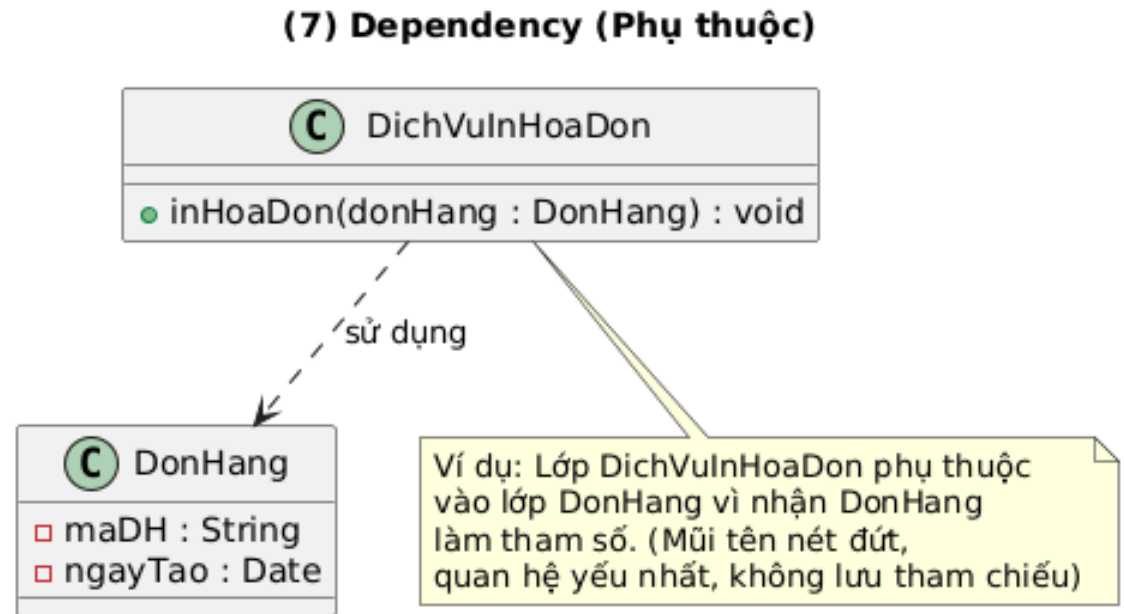


3. Sơ đồ lớp tổng quát (UML Class Diagram)

3.4. Các loại quan hệ giữa các lớp

(7) Dependency (Phụ thuộc):

- ✓ Quan hệ yếu nhất: lớp A sử dụng lớp B trong một phương thức nào đó nhưng không lưu trữ tham chiếu lâu dài.
- ✓ **Ký hiệu:** mũi tên nét đứt.
- ✓ **Ví dụ:** Lớp DichVuInHoaDon phụ thuộc vào lớp DonHang vì nhận DonHang làm tham số



3. Sơ đồ lớp tổng quát (UML Class Diagram)

3.5. Các khái niệm nâng cao

(1) Abstract class và Interface:

- ✓ **Lớp trừu tượng (abstract):** tên lớp viết nghiêng, không thể tạo đối tượng trực tiếp, chỉ dùng làm cha.
- ✓ **Interface:** đánh dấu bằng <<interface>> phía trên tên, định nghĩa hợp đồng mà các lớp triển khai phải tuân theo.

(2) Enumeration (Kiểu liệt kê):

- ✓ Lớp đặc biệt có giá trị cố định. Đánh dấu <<enumeration>>.
- ✓ **Ví dụ:** TrangThaiDonHang có các giá trị {DA_TAO, DANG_PHA, HOAN_THANH, DA_HUY}.

3. Sơ đồ lớp tổng quát (UML Class Diagram)

3.5. Các khái niệm nâng cao

(3) Stereotype: Các nhãn đặc biệt đặt trong dấu << >> để bổ sung ý nghĩa cho lớp:

- ✓ <<entity>>: lớp nghiệp vụ, thường được lưu vào DB.
- ✓ <<service>>: lớp cung cấp dịch vụ, thường stateless.
- ✓ <<repository>>: lớp truy cập dữ liệu, là cầu nối giữa entity và DB.
- ✓ <<controller>>: lớp xử lý request từ giao diện.
- ✓ <<DTO>>: Data Transfer Object, dùng để truyền dữ liệu giữa các tầng.

4. Mô hình quan hệ thực thể (ERD)

4.1. Giới thiệu

- ❑ Mô hình thực thể - quan hệ (Entity-Relationship Diagram – ERD) được Peter Chen đề xuất năm 1976, là công cụ chuẩn để mô hình hóa dữ liệu ở mức quan niệm và logic.
- ❑ Khác với Class Diagram nhấn mạnh vào hành vi (methods), ERD chỉ quan tâm đến cấu trúc dữ liệu – đây là điểm khác biệt quan trọng.

4. Mô hình quan hệ thực thể (ERD)

4.2. Các thành phần cơ bản của ERD

- ☐ Thực thể (Entity)
- ☐ Thuộc tính (Attribute)
- ☐ Quan hệ (Relationship)

4. Mô hình quan hệ thực thể (ERD)

4.2. Các thành phần cơ bản của ERD

❑ Thực thể (Entity)

- ✓ Là một đối tượng trong thế giới thực mà chúng ta cần lưu thông tin.
- ✓ **Ký hiệu:** hình chữ nhật với tên thực thể bên trong.
- ✓ Tên thực thể nên viết ở dạng số ít, danh từ (ví dụ: KHACH_HANG, DON_HANG, NHAN_VIEN, MON).
- ✓ Phân biệt với Class Diagram: thực thể trong ERD tương đương với entity class trong code, không bao gồm các lớp service, controller, DTO.

4. Mô hình quan hệ thực thể (ERD)

4.2. Các thành phần cơ bản của ERD

- ❑ **Thuộc tính (Attribute)** Là đặc điểm của thực thể. Các loại thuộc tính:
- ✓ Thuộc tính **khóa chính** (Primary Key – PK): xác định duy nhất một bản ghi. Thường gạch chân hoặc đánh dấu PK.
 - ✓ Thuộc tính **khóa ngoại** (Foreign Key – FK): tham chiếu đến PK của thực thể khác.
 - ✓ Thuộc tính đơn (Simple): không thể chia nhỏ (vd: tuổi).
 - ✓ Thuộc tính phức (Composite): có thể tách nhỏ (vd: địa chỉ = số nhà + đường + phường + quận).
 - ✓ Thuộc tính đa trị (Multi-valued): có thể có nhiều giá trị (vd: số điện thoại).
 - ✓ Thuộc tính dẫn xuất (Derived): tính được từ thuộc tính khác (vd: tuổi tính từ ngày sinh).

4. Mô hình quan hệ thực thể (ERD)

4.2. Các thành phần cơ bản của ERD

- ❑ **Quan hệ (Relationship)** Là liên kết giữa các thực thể. Mỗi quan hệ có:
 - ✓ Tên quan hệ (động từ): vd: “đặt”, “thuộc về”, “chứa”.
 - ✓ Bậc (degree): nhị phân (binary – 2 thực thể), tam phân (ternary – 3 thực thể)... Trong thực tế hầu như chỉ dùng nhị phân.
 - ✓ Lực lượng (cardinality): 1-1, 1-N, N-N.

4. Mô hình quan hệ thực thể (ERD)

4.3. Ký hiệu Crow's Foot

Ký hiệu ở đầu đường	Đọc là	Ý nghĩa
(vạch đứng)	đúng một	Một và chỉ một
o (vòng tròn)	không hoặc	Tùy chọn (zero)
chân quạ (><)	nhiều	Nhiều (không xác định được tối thiểu là 0 hay 1)
o<	không hoặc nhiều	Tùy chọn, có thể nhiều
<	một hoặc nhiều	Bắt buộc, ít nhất một

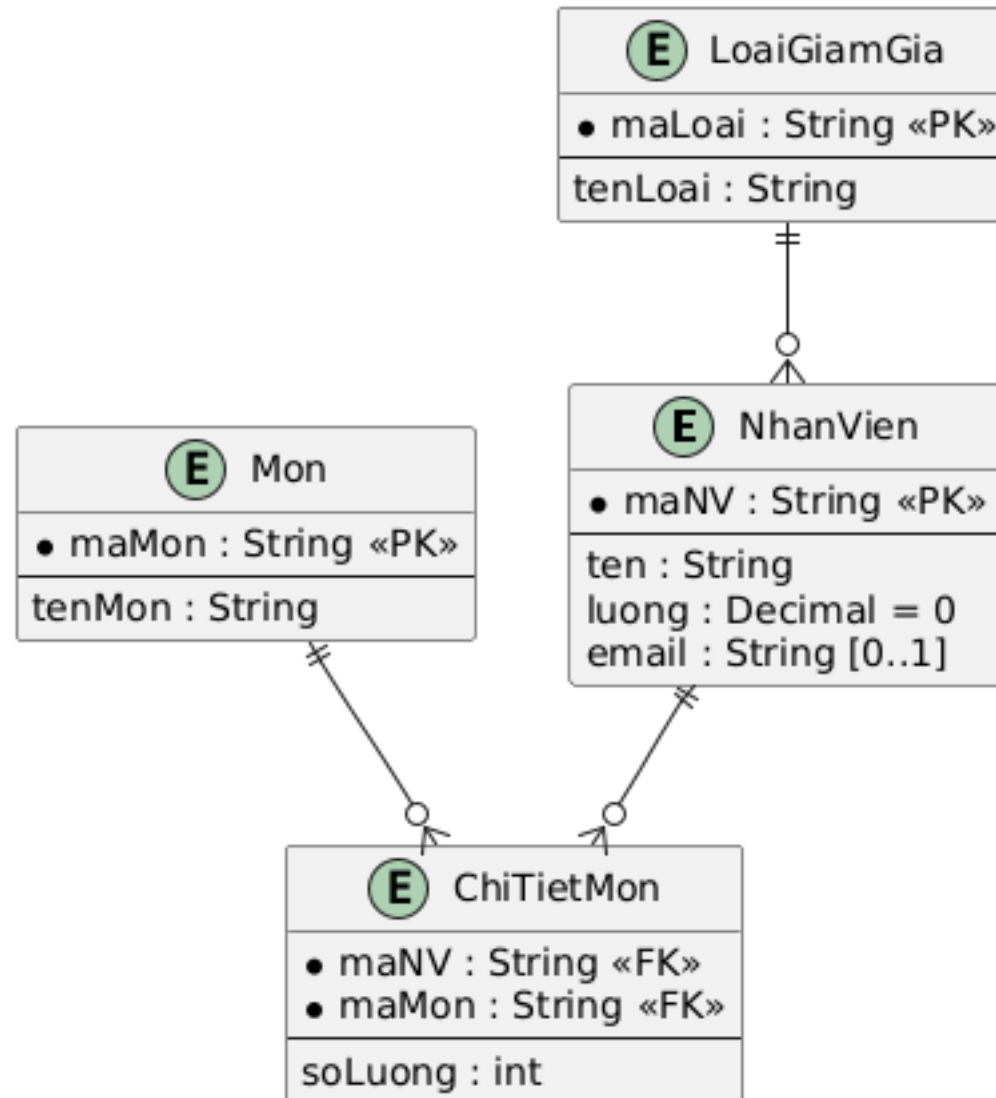
KHACH_HANG —||—————o<— DON_HANG

Đọc: “Một KHACH_HANG có thể đặt không hoặc nhiều DON_HANG; mỗi DON_HANG thuộc về đúng một KHACH_HANG.”

4. Mô hình quan hệ thực thể (ERD)

4.3. Ký hiệu Crow's Foot

Ví dụ:



4. Mô hình quan hệ thực thể (ERD)

4.4. Các loại quan hệ phổ biến

❑ Quan hệ Một - Một (1-1)

- ✓ Mỗi bản ghi của thực thể A liên kết với tối đa một bản ghi của thực thể B và ngược lại.
- ✓ Ví dụ trong Quán Café: Mỗi NHAN_VIEN có duy nhất một TAI_KHOAN đăng nhập.
- ✓ Trong thực tế, quan hệ 1-1 ít gặp và thường được hợp nhất thành một bảng (trừ khi có lý do về bảo mật hoặc hiệu năng).

4. Mô hình quan hệ thực thể (ERD)

4.4. Các loại quan hệ phổ biến

❑ Quan hệ Một - Nhiều (1-N)

- ✓ Phổ biến nhất. Mỗi bản ghi của A liên kết với nhiều bản ghi của B, nhưng mỗi bản ghi của B chỉ thuộc về một bản ghi của A.
- ✓ **Ví dụ:** Một KHACH_HANG có nhiều DON_HANG, mỗi DON_HANG thuộc về một KHACH_HANG.
- ✓ **Cài đặt:** Đặt khóa ngoại (FK) ở thực thể “nhiều”. Trong ví dụ trên, bảng DON_HANG có cột kháchHang_id.

4. Mô hình quan hệ thực thể (ERD)

4.4. Các loại quan hệ phổ biến

❑ Quan hệ Nhiều - Nhiều (N-N)

- ✓ Mỗi bản ghi của A liên kết với nhiều bản ghi của B và ngược lại.
- ✓ **Ví dụ:** Một DON_HANG chứa nhiều MON, một MON có thể xuất hiện trong nhiều DON_HANG.
- ✓ **Cài đặt:** Bắt buộc tạo bảng trung gian (junction table) với hai khóa ngoại.

4. Mô hình quan hệ thực thể (ERD)

4.4. Các loại quan hệ phổ biến

❑ Quan hệ Nhiều - Nhiều (N-N)

✓ Bảng trung gian

```
CHI_TIET_DON_HANG
- donHang_id (FK → DON_HANG)
- mon_id (FK → MON)
- soLuong : int
- donGiaTaiThoiDiem : decimal
- PRIMARY KEY (donHang_id, mon_id)
```

Lưu ý: bảng trung gian thường có thêm các thuộc tính riêng (như số lượng, đơn giá tại thời điểm bán). Đây là lý do quan trọng khiến nó không thể bị bỏ qua.

4. Mô hình quan hệ thực thể (ERD)

4.5. Các mức của ERD

Mức	Đặc điểm	Đối tượng
Conceptual ERD	Chỉ có thực thể và quan hệ chính, không có thuộc tính chi tiết	Ngh nghiệp vụ, khách hàng
Logical ERD	Đầy đủ thuộc tính, khóa chính/ngoại, lực lượng, nhưng chưa có kiểu dữ liệu DB cụ thể	Phân tích viên, kiến trúc sư
Physical ERD	Đã có kiểu dữ liệu, ràng buộc, chỉ mục cụ thể của hệ quản trị	Lập trình viên, DBA

5. Mối liên hệ giữa Class Diagram và ERD

Tiêu chí	Class Diagram	ERD
Nguồn gốc	UML (1990s, OMG)	Peter Chen (1976)
Mục đích chính	Mô hình hóa hướng đối tượng cho code	Mô hình hóa dữ liệu cho cơ sở dữ liệu
Phạm vi	Toàn bộ hệ thống (entity, service, controller...)	Chỉ dữ liệu cần lưu trữ
Hành vi (methods)	Có (thuộc tính + phương thức)	Không (chỉ thuộc tính)
Quan hệ	Association, Aggregation, Composition, Inheritance, Realization	Chỉ có liên kết (1-1, 1-N, N-N)
Đa kế thừa	Có hỗ trợ (qua interface)	Không trực tiếp hỗ trợ
Mức trừu tượng	Linh hoạt, có thể từ rất trừu tượng đến rất cụ thể	Có ba mức rõ ràng (Conceptual/Logical/Physical)
Bảng trung gian N-N	Không cần (dùng List/Set)	Bắt buộc tạo bảng riêng

5. Mối liên hệ giữa Class Diagram và ERD

- ❑ **Dùng Class Diagram khi:** Thiết kế kiến trúc phần mềm hướng đối tượng, mô tả mối quan hệ giữa các thành phần code (entity, service, repository...), tài liệu cho lập trình viên, sinh code tự động.
- ❑ **Dùng ERD khi:** Thiết kế cơ sở dữ liệu, làm việc với DBA hoặc khách hàng (vì ERD dễ hiểu hơn), sinh schema DB tự động qua công cụ.
- ❑ Trong dự án thực tế, hai sơ đồ này thường được dùng song song và bổ trợ cho nhau:
 - ✓ Bắt đầu với ERD ở mức Conceptual để thống nhất domain với khách hàng.
 - ✓ Mở rộng thành Class Diagram domain model để thiết kế code.
 - ✓ Tinh chỉnh ERD lên mức Logical, sau đó Physical để triển khai DB.
 - ✓ Class entity trong code thường khớp với entity trong ERD (qua ORM).

THỰC HÀNH

□ Hướng dẫn:

- ✓ Mỗi nhóm mở lại tài liệu use case diagram và mô tả nghiệp vụ.
- ✓ Sử dụng kỹ thuật “gạch chân danh từ”: đọc kỹ mô tả, gạch chân các danh từ chính có vẻ là thực thể/class nghiệp vụ. (Đọc thêm hướng dẫn)
- ✓ Loại bỏ các danh từ không phải thực thể/class (vd: “giao diện”, “nút bấm”, “hệ thống”).
- ✓ Nhóm các danh từ tương tự lại (vd: “khách”, “khách hàng” → một thực thể/class).
- ✓ Liệt kê các thực thể/class vào bảng kèm theo các thuộc tính dự kiến.

THỰC HÀNH

❑ Bài tập 1: Xác định thực thể/class

- ✓ Xác định ít nhất 7 lớp.
- ✓ Mỗi lớp có các thuộc tính và phương thức dự kiến.

❑ Bài tập 2: Vẽ Class Diagram

- ✓ Xác định quan hệ (Association / Aggregation / Composition / Inheritance)
- ✓ Vẽ Class Diagram trên draw.io với đủ 3 ngăn, multiplicity, loại quan hệ

❑ Bài tập 3: Mapping Class Diagram → ERD Logical

- ✓ Từ Class Diagram vừa vẽ, áp dụng quy tắc mapping để tạo ERD
- ✓ Xử lý kế thừa, quan hệ N-N, Composition